COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/22/26	**DATE SUBMITTED:**	07/22/26
NAME:	MATT JOHN ACALA	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. 3 **Creating a Singly Linked List**

1. PROCEDURE OUTPUT

Node Definition:

```
template <typename T> //node creation
class SingleList{
    public:
    T data; //This contains the head of the node
    SingleList<T>* next; //This contains the address of the next node
};
```

Traversal:

```
template <typename T> //Traversal
    void ListTraversal (SingleList<T>* head) {

        if(head == nullptr) {
            std::cout << "List is empty." << std::endl;
            return;
        }

        while (head != nullptr) { //Print all data until it reaches nullptr
            std::cout << head->data << " ";
            head = head->next;
        }
        std::cout << std::endl;
    }
```

Types of Insertion:

Head Insertion

```cpp
//Insertion Head
template <typename T>
void InsertAtHead(SingleList<T>*& head, T value) {
    SingleList<T>* newNode = new SingleList<T>();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}
```

Insertion at Specific position

```cpp
//Insertion at a specific position
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode)
{
    // 1. Check if previous node is null
    if (prevNode == nullptr) {
        std::cout << "Previous value cannot be null!" << std::endl;
        return;
    }

    // 2. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 3. Store data in the new node
    newNode->data = newData;

    // 4. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;

    // 5. Point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at tail

```cpp
//Insertion Tail
template <typename T>
void InsertAtTail(SingleList<T>*& head, T value) {
    SingleList<T>* newNode = new SingleList<T>();
    newNode->data = value;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    } else {
        SingleList<T>* current = head;
        while (current->next != nullptr) {
            current = current->next;
        }
        current->next = newNode;
    }
}
```

2 Type of deletion:

Delete Node

```cpp
//Delete node
template <typename T>
void DeleteNode(SingleList<T>*& head, T value) {
    if (head == nullptr) {
        return; // if List is empty
    }

    SingleList<T>* temp = head;
    SingleList<T>* prev = nullptr;

    //search for the node to be deleted
    while (temp != nullptr && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }

    // if the node was not found
    if (temp == nullptr) {
        return; // Node with the given value not found
    }

    // if Node found delete it
    if (prev == nullptr) {
        head = head->next;
    } else {
        prev->next = temp->next;
    }
    delete temp;
}
```

Delete List

```cpp
template <typename T>
void DeleteList(SingleList<T>*& head) {
    SingleList<T>* current = head;
    SingleList<T>* next = nullptr;

    while (current != nullptr) {
        next = current->next;
        delete current;
        current = next;
    }

    head = nullptr; // Mark the list as empty
}
```

Main.cpp

```cpp
#include <iostream>
#include "singly_ll.h"




int main() {

    // Create the head of the list
    SingleList<int>* head = new SingleList<int>();
    SingleList<int>* second = new SingleList<int>();

    //storing a value inside of the nodes
    head->data = 5;
    second->data = 10;


    //assigning the next pointers
    head->next = second;
    second->next = nullptr;

    std::cout << "Initial List: ";
    ListTraversal(head);

    std::cout << "Trying insert at head: ";
    InsertAtHead(head, 3);
    ListTraversal(head);

    std::cout << "Trying insert at tail: ";
    InsertAtTail(head, 12);
    ListTraversal(head);

    std::cout << "Trying insert at position 1: ";
    sllGeneralInsert(8, head->next);
    ListTraversal(head);

    std::cout << "Trying delete node with value 10: ";
    DeleteNode(head, 10);
    ListTraversal(head);

    std::cout << "Trying delete list: ";
    DeleteList(head);
    ListTraversal(head);

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Programming\C++\SCHOOL\CPE 010> cd "c:\
Initial List: 5 10
Trying insert at head: 3 5 10
Trying insert at tail: 3 5 10 12
Trying insert at position 1: 3 5 8 10 12
Trying delete node with value 10: 3 5 8 12
Trying delete list: List is empty.
PS C:\Programming\C++\SCHOOL\CPE 010>
```

Reflection/Observation:

This linked list header show how templates make structures reusable across types, while pointer manipulation enable dynamic insertion, deletion, and traversal. It also highlight the importance of handling edge cases like empty lists or head operations to keep correctness. Overall, it is a practical exercise in balancing flexibility, memory safety, and efficiency

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CoPilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Example generator and Concept explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I also used it to explain the difference of my code to Engr. Jimlord code since i change a little bit in my code**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.